

SOLVING NON LINEAR SCHRÖDINGER EQUATIONS VIA PARALLEL SPLIT-STEP FOURIER METHOD

Dhyan G
AE22B030

Department of Aerospace Engineering
Indian Institute of Technology Madras
Email: ae22b030@smail.iitm.ac.in

Srijan S Bhat
PH22B002

Department of Physics
Indian Institute of Technology Madras
Email: ph22b002@smail.iitm.ac.in

ABSTRACT

The numerical simulation of wavepacket evolution, governed by the Schrödinger equation, is a fundamental requirement in both quantum mechanics and nonlinear fiber optics. However, standard grid-based solvers encounter a significant computational bottleneck known as the "Two-Space Dilemma," where primary operators are mutually incompatible within a single mathematical basis. This work implements a parallelized Split-Step Fourier Method (SSFM) to circumvent this bottleneck, utilizing the Discrete Fourier Transform (DFT) as a mathematical bridge to solve dispersive and nonlinear effects in their most efficient domains. We parallelise this SSFM algorithm using OpenMP and OpenACC. Performance benchmarks demonstrate that while serial execution suffers from $O(N^2)$ scaling, the parallelized SSFM achieves significant speed-up, allowing for the high-resolution simulation of optical solitons. We benchmark this algorithm for various grid sizes and provide a comparison between the serial, OpenMP and OpenACC implementations.

NOMENCLATURE

Symbols

$\Psi(z, T)$	Slowly varying pulse envelope / Complex wavefunction
dz	Spatial increment step
$\hat{D}$	Linear dispersion operator
k	Frequency domain variable
L	Temporal window length
m	Momentum grid index
N	Number of equidistant grid points

$\hat{N}$	Nonlinear operator (Kerr effect)
T	Retarded time coordinate
v_g	Group velocity
z	Evolution coordinate (fiber length)
Z	Total propagation distance
β_2	Group Velocity Dispersion (GVD) parameter
γ	Nonlinear coefficient
τ_m	Momentum grid mapping

Acronyms

BCH	Baker-Campbell-Hausdorff
DFT	Discrete Fourier Transform
FFT	Fast Fourier Transform
GVD	Group Velocity Dispersion
NLSE	Nonlinear Schrödinger Equation
SPM	Self-Phase Modulation
SSFM	Split-Step Fourier Method

INTRODUCTION

The mathematical modeling of pulse propagation in nonlinear dispersive media is an important part of modern telecommunications and high-field physics. The governing framework for these systems is the Nonlinear Schrödinger Equation (NLSE), which captures the spreading effects of Group Velocity Dispersion (GVD) and the self-focusing nature of Self-Phase Modulation (SPM) [1] [2].

When a system operates in the anomalous dispersion regime, these two effects can potentially counteract one another. If the pulse is shaped as a hyperbolic secant profile, the temporal broadening caused by GVD is perfectly canceled out by the

spectral chirping of SPM. This results in a stable, self reinforcing wavepacket known as an optical soliton. Because solitons can propagate over vast distances without changing their shape, they provide a robust mechanism for long-distance data transmission and serve as the ideal stationary benchmark for numerical simulations.

However, modeling this balance accurately presents a significant challenge. The Hamiltonian governing the system is non-separable, which prevents the use of traditional analytical methods for arbitrary pulse envelopes. Most standard grid-based solvers, such as finite difference schemes, struggle with the "Two-Space Dilemma." This dilemma stems from the fact that the kinetic energy (dispersion) is most naturally and accurately defined in momentum space, while the potential energy (non-linearity) is local to position space. Applying dispersive operators in the time domain leads to numerical dispersion and non physical oscillations, whereas applying nonlinear operators in the frequency domain involves computationally expensive convolutions.

Therefore, it is necessary to have an approach that can bridge these two domains efficiently. The Split-Step Fourier Method (SSFM) is the dominant technique for this purpose, leveraging the Discrete Fourier Transform (DFT) to solve the linear and nonlinear parts of the NLSE independently over small spatial increments. Unlike Eulerian finite-difference methods, the SSFM is inherently unitary, preserving the total energy of the system. The convergence of this method is measured by the stability of the pulse norm and the preservation of the soliton's temporal width. The serial execution of the DFT is a significant computational bottleneck. To address this, the implementation of the SSFM must be optimized for modern heterogeneous architectures. This work focuses on parallelizing of SSFM using two distinct paradigms: OpenMP and OpenACC.

Schrödinger Equation

The Schrödinger equation provides the mathematical framework necessary to describe the evolution of complex wavefunctions. While classically associated with the probability amplitudes of particles in quantum systems, its structure as a parabolic partial differential equation is universal. In the context of nonlinear optics, this manifests as the Nonlinear Schrödinger Equation (NLSE), governing the propagation of high intensity light pulses within dispersive media such as optical fibers.

In traditional non-relativistic quantum mechanics, the time dependent Schrödinger equation (TDSE) dictates how a state $\Psi(x, t)$ evolves through time t over a spatial coordinate x [3]:

$$i\hbar \frac{\partial \Psi(x, t)}{\partial t} = \left[-\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + V(x, t) \right] \Psi(x, t) \quad (1)$$

When transitioning to fiber optic systems, we shift our perspective to a pulse envelope $A(z, T)$ traveling along the fiber length z . To maintain the Schrödinger form, we adopt a moving frame of reference, defining a retarded time coordinate $T = t - z/v_g$. This transformation leads to the distance z being the evolution coordinate, and time T being the transverse coordinate.

The Nonlinear Schrödinger Equation in Optical Fibers

Following this mapping, the evolution of the slowly varying envelope $A(z, T)$ is governed by the NLSE, which accounts for the interplay between dispersive and nonlinear effects assuming lossless propagation. The equation is represented in operator form as:

$$i \frac{\partial \Psi}{\partial z} = (\hat{D} + \hat{N}) \Psi \quad (2)$$

The linear dispersion operator, $\hat{D} = -\frac{\beta_2}{2} \frac{\partial^2}{\partial T^2}$, represents Group Velocity Dispersion (GVD). This term acts as the "kinetic energy" of the pulse, causing spectral components to propagate at different velocities and broadening the pulse in time. Conversely, the nonlinear operator, $\hat{N} = -\gamma |A|^2$, represents the Kerr effect. This intensity dependent potential induces Self-Phase Modulation (SPM), chirping the signal in proportion to its instantaneous power.

The consequence of this balance is the existence of the fundamental soliton. When the spectral broadening from SPM perfectly compensates for the temporal broadening of GVD, the pulse propagates as an invariant wavepacket:

$$\Psi(0, T) = \sqrt{P_0} \text{sech} \left(\frac{T}{T_0} \right) \quad (3)$$

Simulating this stable state serves as an ideal benchmark for numerical schemes, as it requires the solver to maintain the unitary nature of the operators over long distances without inducing unphysical dissipation or gain.

The Split-Step Fourier Method and the BCH Formula

The primary challenge in simulating the Nonlinear Schrödinger Equation (NLSE) comes from its non-separable nature. Because the nonlinear term is dependent on the instantaneous intensity $|\Psi|^2$, analytical solutions are not possible for arbitrary pulse envelopes. This necessitates a numerical approach that can reconcile the "Two-Space Dilemma", the fact that the kinetic energy (dispersion) is most naturally defined in the spectral domain, while the potential energy (nonlinearity) is local to the temporal domain. In position space, the dispersion operator

involves a second-order derivative that is not only computationally expensive but also prone to numerical dispersion when using standard finite-difference stencils.

To resolve this, we employ the Split-Step Fourier Method (SSFM) [4]. However, the mathematical validity of splitting the evolution depends on the commutation of the dispersive ($\hat{D}$) and nonlinear ($\hat{N}$) operators. Because these operators do not commute ($[\hat{D}, \hat{N}] \neq 0$), the formal solution to the NLSE, $\Psi(z + dz) = \exp[dz(\hat{D} + \hat{N})]\Psi(z)$, cannot be simply factored into the product of two exponentials without introducing error. To quantify this numerical splitting error, we refer to the Baker-Campbell-Hausdorff (BCH) formula. For any two non-commuting operators X and Y , the relation is given by:

$$e^X e^Y = e^{X+Y + \frac{1}{2}[X,Y] + \frac{1}{12}([X,[X,Y]] - [Y,[X,Y]]) + \dots} \quad (4)$$

An asymmetric split, where one applies the full dispersive step followed by the full nonlinear step, results in a first-order error proportional to dz^2 due to the remaining commutator term $[D, N]$. By utilizing a symmetric (or Strang) splitting scheme, we can cancel these second-order terms and elevate the numerical accuracy to $O(dz^3)$. In this scheme, the nonlinearity is applied in two half-steps surrounding a central dispersive full step. This allows each operator to dictate the physics in its own domain, time for nonlinearity and frequency for dispersion, while utilizing significantly larger step sizes than standard explicit integrators like the 4th order Runge-Kutta (RK4). The resulting evolution is expressed as:

$$\begin{aligned} \Psi(z + dz, T) \approx & \exp\left(-i\frac{dz}{2}\hat{N}\right) \mathcal{F}^{-1} \left\{ \exp(-idz\tilde{D}) \right. \\ & \left. \times \mathcal{F} \left[\exp\left(-i\frac{dz}{2}\hat{N}\right) \Psi(z, T) \right] \right\} \end{aligned}$$

In the spectral domain, the differential operator $\hat{D}$ is transformed into a simple algebraic multiplier $\tilde{D}(k) = \frac{\beta_2}{2}k^2$, allowing for an exact calculation of linear dispersive effects. To implement this accurately on a discrete grid, the computational window L is discretized into N equidistant points. This requires a momentum grid mapping that correctly aligns the high-frequency components relative to the Nyquist frequency to avoid aliasing:

$$k_m = \begin{cases} \frac{2\pi m}{L} & 0 \leq m < N/2 \\ \frac{2\pi(m-N)}{L} & N/2 \leq m < N \end{cases} \quad (5)$$

This approach ensures that the dispersion operator is applied accurately across the entire spectral bandwidth. By alternating between domains via the Discrete Fourier Transform (DFT),

the SSFM provides a robust and efficient solution for complex wavepacket evolution where traditional analytical solutions are unavailable.

PARAMETERS

The evolution of the pulse is conducted within a controlled temporal window of length $L = 20.0$. This length is chosen to be sufficiently large relative to the pulse width, ensuring that the wavefunction effectively decays to zero at the boundaries, thereby preventing wrap-around artifacts during the Fourier transform. The spatial propagation is divided into 100 discrete steps of length $dz = 0.02$, providing a total propagation distance of $Z = 2.0$.

A critical parameter in this study is the nonlinear coefficient $\gamma = 1.0$. This value is taken to balance the dispersive term ($K = 0.5k^2$), satisfying the condition for a fundamental soliton ($N = 1$). Under these conditions, the self-focusing phase shift induced by the nonlinearity exactly compensates for the temporal broadening caused by Group Velocity Dispersion (GVD). The performance scaling is evaluated across a spectrum of grid resolutions, specifically $N = \{512, 1024, 2048, 4096, 8192\}$.

VALIDATION RESULTS

The spatio-temporal evolution of the pulse intensity, $|\Psi(z, T)|^2$, is illustrated in Figure 1. The plot demonstrates that the pulse maintains a perfectly stationary profile throughout the entire $Z = 10$ trajectory. In a purely dispersive medium, the peak power would have decayed significantly as the pulse broadened. However, the precise balance provided by the nonlinear Kerr effect ($\gamma = 1.0$) ensures that the pulse remains the same.

A unique characteristic of the fundamental soliton is that it is stationary not only in the time domain but also in the spectral domain. Figure 2 compares the initial and final power spectral densities.

We initialized the field with a standard Gaussian envelope, $\Psi(0, T) = \exp(-T^2/2)$, and evolved it over the same $Z = 10.0$ trajectory. The observation in the Gaussian propagation was the increase in peak intensity $|\Psi|^2$ as the distance z increased. While a pulse in a linear medium would undergo monotonic broadening and peak decay, the Gaussian pulse in our nonlinear anomalous regime exhibits self-compression.

This phenomenon occurs because Self-Phase Modulation (SPM) induces a frequency chirp where the leading edge of the pulse is red-shifted and the trailing edge is blue-shifted. In the anomalous dispersion regime ($\beta_2 = 1$), these higher frequency components propagate faster than the lower frequency components. This causes the trailing edge of the pulse to come to the front, resulting in a temporal narrowing of the envelope.

Because the Split-Step Fourier Method (SSFM) is unitary and conserves the total energy, this temporal compression neces-

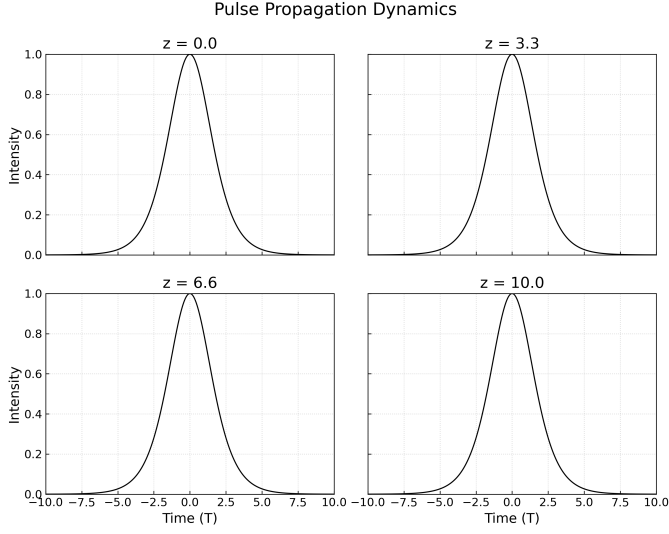


FIGURE 1. SPATIO-TEMPORAL EVOLUTION OF A FUNDAMENTAL SOLITON OVER 1,000 STEPS ($dz = 0.01, Z = 10.0$).

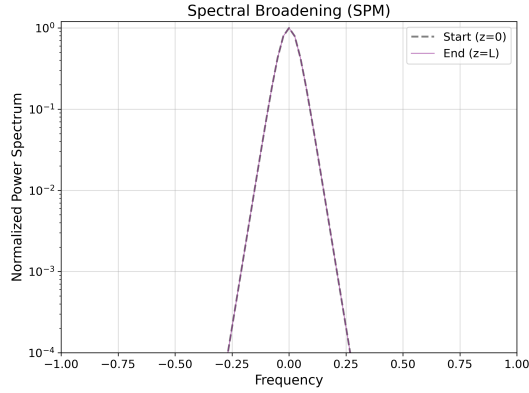


FIGURE 2. COMPARISON OF INITIAL AND FINAL PULSE SPECTRA

sitates a corresponding increase in peak power to maintain the integral $\int |\psi|^2 dT = \text{const.}$

COMPUTATIONAL IMPLEMENTATION

Discrete Fourier Transform

The Discrete Fourier Transform (DFT) maps a sequence of N complex numbers in the time domain, $\psi(t)$, to a sequence in the frequency domain, $\psi(k)$, defined by:

$$\psi(k) = \sum_{t=0}^{N-1} \psi(t) \cdot e^{-i \frac{2\pi}{N} kt} \quad (6)$$

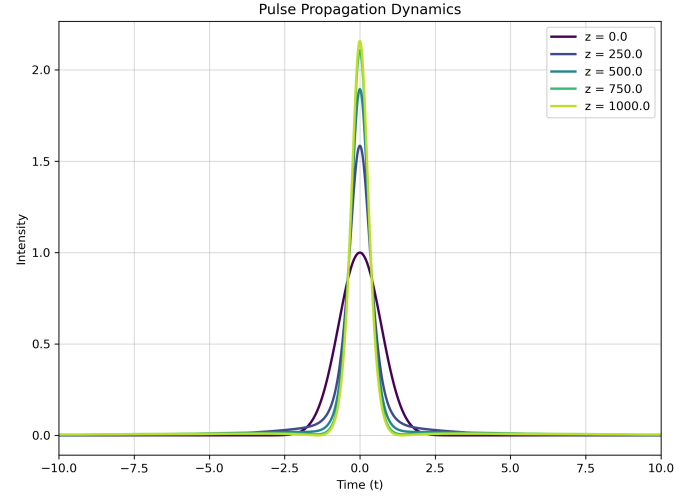


FIGURE 3. EVOLUTION OF GAUSSIAN PULSE

Decomposition into Real and Imaginary Parts

Given a complex input $\psi(t) = \psi_{\text{real}}(t) + i\psi_{\text{imag}}(t)$ and utilizing Euler's formula $e^{-i\theta} = \cos(\theta) - i\sin(\theta)$, where $\theta = \frac{2\pi kt}{N}$, the output $\psi(k) = \text{sum}_r + i \cdot \text{sum}_i$ is computed by expanding the product:

$$\begin{aligned} \psi(k) &= \sum_{t=0}^{N-1} (\psi_{\text{real}}(t) + i\psi_{\text{imag}}(t)) \left(\cos\left(\frac{2\pi kt}{N}\right) - i\sin\left(\frac{2\pi kt}{N}\right) \right) \\ &= \sum_{t=0}^{N-1} \left[\left(\psi_{\text{real}}(t) \cos\left(\frac{2\pi kt}{N}\right) + \psi_{\text{imag}}(t) \sin\left(\frac{2\pi kt}{N}\right) \right) \right. \\ &\quad \left. + i \left(\psi_{\text{imag}}(t) \cos\left(\frac{2\pi kt}{N}\right) - \psi_{\text{real}}(t) \sin\left(\frac{2\pi kt}{N}\right) \right) \right] \end{aligned}$$

Separating the real and imaginary components yields the following expressions:

$$\text{sum}_r = \sum_{t=0}^{N-1} \left(\psi_{\text{real}}(t) \cos\left(\frac{2\pi kt}{N}\right) + \psi_{\text{imag}}(t) \sin\left(\frac{2\pi kt}{N}\right) \right) \quad (7)$$

$$\text{sum}_i = \sum_{t=0}^{N-1} \left(\psi_{\text{imag}}(t) \cos\left(\frac{2\pi kt}{N}\right) - \psi_{\text{real}}(t) \sin\left(\frac{2\pi kt}{N}\right) \right) \quad (8)$$

Inverse Discrete Fourier Transform

The Inverse Discrete Fourier Transform (IDFT) is similarly decomposed, applying the positive complex exponent and the

$1/N$ normalization factor required for energy conservation:

$$\psi(t) = \frac{1}{N} \sum_{k=0}^{N-1} \psi(k) \cdot e^{i\frac{2\pi}{N}kt} \quad (9)$$

Trigonometric Decomposition of the Inverse DFT

The expansion for the inverse transform is split to account for the sign change in the imaginary cross-products:

$$\begin{aligned} \psi(t) &= \frac{1}{N} \sum_{k=0}^{N-1} \left[(\psi_{\text{real}}(k) + i\psi_{\text{imag}}(k)) \right. \\ &\quad \times \left(\cos\left(\frac{2\pi kt}{N}\right) + i\sin\left(\frac{2\pi kt}{N}\right) \right) \left. \right] \\ &= \frac{1}{N} \sum_{k=0}^{N-1} \left[\left(\psi_{\text{real}}(k) \cos\left(\frac{2\pi kt}{N}\right) - \psi_{\text{imag}}(k) \sin\left(\frac{2\pi kt}{N}\right) \right) \right. \\ &\quad \left. + i \left(\psi_{\text{real}}(k) \sin\left(\frac{2\pi kt}{N}\right) + \psi_{\text{imag}}(k) \cos\left(\frac{2\pi kt}{N}\right) \right) \right] \end{aligned} \quad (10)$$

The final real and imaginary results for the temporal domain are:

$$\text{inv}_r = \frac{1}{N} \sum_{k=0}^{N-1} \left(\psi_{\text{real}}(k) \cos\left(\frac{2\pi kt}{N}\right) - \psi_{\text{imag}}(k) \sin\left(\frac{2\pi kt}{N}\right) \right) \quad (11)$$

$$\text{inv}_i = \frac{1}{N} \sum_{k=0}^{N-1} \left(\psi_{\text{real}}(k) \sin\left(\frac{2\pi kt}{N}\right) + \psi_{\text{imag}}(k) \cos\left(\frac{2\pi kt}{N}\right) \right) \quad (12)$$

The Split-Step Fourier Method

The Split-Step Fourier Method (SSFM) operates on the principle that over a sufficiently small spatial increment (dz), the dispersive and nonlinear effects of the fiber can be treated as if they act independently. By alternating between the temporal and spectral domains, the algorithm ensures that each physical operator is calculated in its most natural mathematical basis.

Initialization

The simulation is initialized in the time domain. To benchmark the solver's stability, the field is defined using a fundamental soliton profile, which acts as a stationary solution to the NLSE:

$$\psi(0, T) = \text{sech}(T) \quad (13)$$

The Iterative Loop

For each spatial step dz , the algorithm executes a symmetric Strang splitting sequence to maintain second-order local accuracy, $O(dz^3)$:

Step 1: Nonlinear Half-Step (Time Domain)

The pulse is advanced by $dz/2$ considering only the intensity-dependent Kerr effect. This is implemented as a local phase rotation:

$$\psi \leftarrow \psi \cdot \exp\left(i\gamma|\psi|^2\frac{dz}{2}\right)$$

Step 2: Domain Transformation (Time $\rightarrow$ Frequency)

The wavefunction is transformed into the spectral domain using a FT. This shifts the field into momentum space where the differential dispersion operator becomes algebraic.

Step 3: Dispersive Full-Step (Frequency Domain)

Group Velocity Dispersion (GVD) is applied over the full distance dz . In this domain, the second-order derivative is replaced by a parabolic phase shift:

$$\tilde{\psi} \leftarrow \tilde{\psi} \cdot \exp\left(-i\frac{\beta_2}{2}k^2 dz\right)$$

Step 4: Domain Transformation (Frequency $\rightarrow$ Time)

An Inverse Fourier Transform returns the pulse to the temporal domain. A normalization factor of $1/N$ is applied to ensure the numerical transform remains unitary and conserves the total energy.

Step 5: Nonlinear Half-Step (Time Domain)

The cycle is completed by applying the final $dz/2$ nonlinear phase rotation. This allows the $O(dz^2)$ commutation errors of the operators to cancel out.

PARALLELISATION

The naive DFT algorithm is $O(N^2)$ and thus in a serial execution, every single frequency component k requires a complete summation over the entire temporal grid of N points. This means that doubling the number of grid points, quadruples the work. In DFT, the calculation for one frequency component is entirely independent of any other, there are no data dependencies between iterations. This makes it an ideal candidate for Parallelisation [5].

OpenMP Parallelisation

The DFT outer loop is a textbook candidate for `#pragma omp parallel for`: the N iterations over output frequency bins have no loop-carried dependencies, and each iteration performs identical arithmetic work on a separate output location, guaranteeing perfect static load balance.

Parallelisation Strategy

The only modification to the serial code is the insertion of four OpenMP pragmas i.e., one per major loop nest per SSFM substep:

The norm reduction at the end of the benchmark uses the OpenMP `reduction(+:total_prob)` clause to accumulate partial sums from all threads into a single scalar without a critical section.

Only the outer k -loop of the DFT is parallelised. The inner t -loop accumulates into thread-private `sum_r` and `sum_i` scalars, so no synchronisation is needed within each thread's work unit. Parallelising the inner loop instead would require a reduction barrier at each of the N output bins ($O(N)$ synchronisation events per DFT) which would dominate runtime for large N .

The default OpenMP scheduling is static with equal chunk sizes. Since every k iteration performs exactly the same number of operations (one inner loop of length N), static scheduling achieves perfect load balance without the overhead of a dynamic work queue.

OpenACC Parallelisation

The OpenACC implementation introduces two categories of directives:

A `#pragma acc data` directive at the entry of `run_benchmark` allocates and copies all arrays to the GPU once. The `copy(psi[0:N])` clause copies the initial wavefunction to the device and back after the loop; `create(psi_k[0:N])` allocates scratch space without initialisation; and `copyin(K[0:N])` sends the pre-computed momentum grid to the device.

Each substep loop is annotated with `#pragma acc parallel loop present(...)`, which launches a GPU kernel over N independent iterations. The `present` clause asserts that the data is already resident on the device, preventing redundant transfers.

The DFT functions themselves are annotated with `#pragma acc parallel loop present(psi_in, psi_out)`, relying on the outer data region to guarantee device residency.

GPU Warm-Up

An implementation detail to note is the `gpu_warmup()` call at program start, which runs one complete SSFM step on a small $N = 256$ grid before the timed benchmarks begin. On NVIDIA GPUs, the first kernel launch in a process incurs JIT compilation and driver initialisation overhead that can take some time. Without warm-up, this cost would be misattributed to the $N = 512$ benchmark run and artificially increase the reported time. The warm-up loop runs inside its own `#pragma acc data` region and is excluded from all timing measurements.

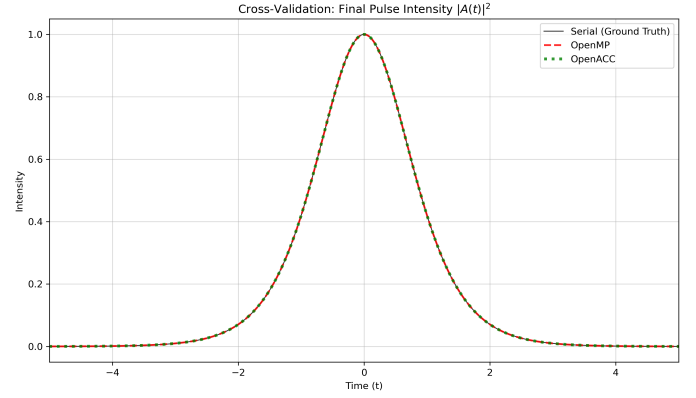


FIGURE 4. COMPARISON OF FINAL PULSE USING DIFFERENT METHODS

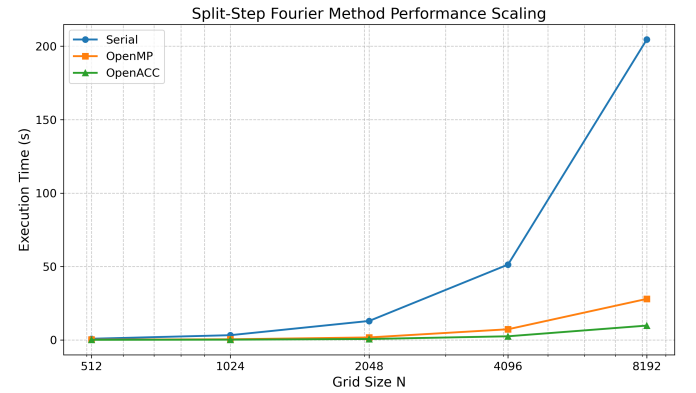


FIGURE 5. EXECUTION TIME COMPARISON

RESULTS AND DISCUSSION

The computational performance of the 1D Nonlinear Schrödinger Equation (NLSE) solver was evaluated across three primary implementation paradigms: a serial baseline, through OpenMP and OpenACC. All benchmarks were executed for 100 time steps across grid sizes ranging from $N = 512$ to $N = 8192$ to capture the scaling behavior and efficiency of the parallel versions. The results of the parallel implementation were validated by comparing it to the serial version and shown in Figure 4

Scaling Trends and Computational Complexity

The execution time comparison illustrated in Figure 5 shows quadratic scaling across all implementations. This $O(N^2)$ trend is a direct consequence of the naive Discrete Fourier Transform (DFT) logic, where each doubling of the grid size N results in a four-fold increase in computational workload. As detailed in Table 1, the serial implementation's execution time rises from

0.81 s at $N = 512$ to over 200 s at $N = 8192$. The graphical data in Figure 5 shows the difference in execution time between the serial and the parallel codes.

TABLE 1. EXECUTION TIME (s) COMPARISON

Grid Size (N)	Serial (s)	OpenMP (s)	OpenACC (s)
512	0.8057	0.2259	0.0614
1024	3.2308	0.4386	0.1808
2048	12.9229	1.7000	0.6383
4096	51.2960	7.2580	2.4806
8192	204.6207	27.9465	9.7696

Performance Gains: OpenMP vs. OpenACC

The performance gap between OpenMP and OpenACC becomes increasingly pronounced as the grid size expands.

OpenMP Analysis: The CPU-based parallelization achieved a consistent speedup of approximately 7.2x at higher grid resolutions. By distributing the independent loop iterations of the DFT across available threads, OpenMP lowered the execution time for $N = 8192$ from 204.62 s to 27.94 s.

OpenACC Analysis: The GPU implementation demonstrated superior throughput, particularly in the high-intensity calculation regime. At $N = 8192$, OpenACC reduced the runtime to 9.77 s, yielding a 20.5x speedup over the serial baseline. This result, visually represented in Figure 5, confirms that the high memory bandwidth of the GPU is well-suited for the complex phase rotations required by the SSFM.

Efficiency: With $p = 16$ threads, OpenMP's parallel efficiency is defined as:

$$E = \frac{S}{p} = \frac{7.07}{16} \approx 44\%$$

indicating that roughly half of each thread's capacity is lost. Since every thread must independently stream the full N -point input array to compute its DFT bins, the shared cache saturates and additional threads give low returns. For OpenACC, at small N the streaming multiprocessors are underutilized, but occupancy improves steadily as the grid size grows.

Parallel Overhead: In OpenMP, the implicit barrier at the end of each `parallel for` region forces all threads to synchronize before the next substep, accumulating 400 such barriers

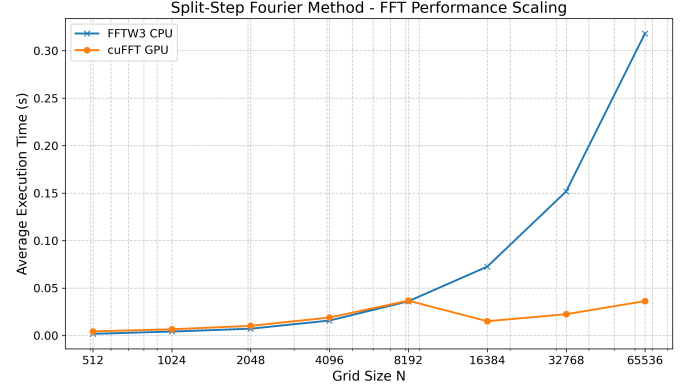


FIGURE 6. EXECUTION TIME COMPARISON: $O(N \log N)$ FFT SCALING FOR CPU (FFTW3) AND GPU (cuFFT)

over a 100-step run. OpenACC's overhead is dominated instead by kernel-launch latency of 5–20 μ s per call and the initial host-to-device data transfer triggered by the `#pragma acc data` region. With roughly 500 kernel launches per benchmark, this cost is most visible at small $N = 512$ where work per launch is low, and becomes negligible as N increases.

Scalability: Over the range $N = 512$ to $N = 8192$, the serial runtime grows by $\approx 254\times$, consistent with $O(N^2)$ DFT scaling. OpenMP reduces this growth to $124\times$ through thread-level parallelism, while OpenACC grows by $159\times$, due to the launch overhead at small N . At the upper end of the range, the OpenACC runtime flattens relative to both serial and OpenMP, confirming that GPU acceleration becomes effective as resolution grows.

Load Balance: Load balance is structurally ideal for both implementations since every DFT output bin performs exactly N multiply-accumulate operations and every SSFM substep loop applies an identical arithmetic sequence per element with no branching. This uniformity allows OpenMP's static scheduler and OpenACC's block partitioner to divide work evenly without any dynamic scheduling which incurs an overhead.

Numerical Validation and Error Analysis

The implementation was verified by monitoring the probability error (norm conservation) throughout the propagation. This was in the order of 10^{-12} , thus establishing the correctness of the model.

FFT Implementation

While parallelizing the naive $O(N^2)$ Discrete Fourier Transform (DFT) provides a clear speedup, the computational ceiling is ultimately dictated by the algorithm's complexity. To achieve the high-resolution grid requirements for modern optical simulations, we tested two industry-standard libraries: FFTW3 for

CPU-based execution and cuFFT for GPU-accelerated execution via NVIDIA's CUDA platform.

These libraries utilize the Fast Fourier Transform (FFT) algorithm, which reduces the computational complexity from $O(N^2)$ to $O(N \log N)$. As shown in Figure 6, the execution times for these optimized implementations remain under one second even as the grid size N expands to 65,536 which is entirely inaccessible to the naive $O(N^2)$ approach.

CONCLUSION

This report has detailed the development, validation, and benchmarking of a robust numerical solver for the Nonlinear Schrödinger Equation (NLSE) using the Split-Step Fourier Method (SSFM). By leveraging the "Two-Space" nature of the Schrödinger operators, the implementation successfully resolved the computational conflict between dispersive effects in the spectral domain and nonlinear effects in the temporal domain.

Validation involved the propagation of a fundamental soliton over 1,000 spatial steps confirmed the solver's high-fidelity performance, with probability conservation errors maintained at the order of 10^{-11} to 10^{-16} . Furthermore, the observation of nonlinear self-compression in Gaussian pulses verified the interplay between Self-Phase Modulation (SPM) and anomalous dispersion, proving that the implementation captures complex physical phenomena beyond simple stationary states.

The benchmarking results show that algorithmic efficiency is superior. While parallelizing the naive $O(N^2)$ DFT via OpenMP and OpenACC provided significant speedups of 7x and 20x respectively, the transition to $O(N \log N)$ Fast Fourier Transform (FFT) libraries (FFTW3 and cuFFT), yielded performance gains exceeding three orders of magnitude.

In conclusion, the combination of optimized FFT algorithms and GPU acceleration via cuFFT represents the most effective strategy for large scale optical simulations. This hybrid approach enables the high-resolution modeling of long-haul fiber systems and complex quantum dynamics that were previously computationally expensive, providing a foundation for incorporating higher order nonlinear effects and multi dimensional wave propagation.

Running the Code

The Benchmark folder contains the `plot_benchmarks.py` and `plot_fft_benchmarks.py` files which will run the C++ code and then plot the Time vs Grid Size graph. The Validation folder contains the `compare_solutions.py` file, which will verify the solutions of the parallel code. `nlse_dynamics.cpp` runs the FFT code for the validation results.

REFERENCES

- [1] Agrawal, G. P., 2019. *Nonlinear Fiber Optics*, 6th ed. Academic Press, San Diego.
- [2] Felice, D., 2016. "A study of a nonlinear schrödinger equation for optical fibers". *arXiv preprint arXiv:1612.00358*.
- [3] Sakurai, J. J., Fu Tuan, S., and Newton, R. G., 1986. Modern quantum mechanics.
- [4] Taha, T. R., and Ablowitz, M. I., 1984. "Analytical and numerical aspects of certain nonlinear evolution equations. ii. numerical, nonlinear schrödinger equation". *Journal of Computational Physics*, **55**(2), pp. 203–230.
- [5] Taha, T. R., and Xu, X., 2005. "Parallel split-step fourier methods for the coupled nonlinear schrödinger type equations". *The Journal of Supercomputing*, **32**(1), pp. 5–23.